

Day 4: Finite State Machines (FSM)

Session 1: FSM Fundamentals

Welcome to the Most Important Topic in Digital Design!

Finite State Machines (FSMs) are EVERYWHERE:

- CPU control units
- Communication protocols (UART, SPI, I²C)
- Game logic and AI
- Traffic lights, elevators, vending machines
- Memory controllers
- ANY system that makes decisions based on current state and inputs

After today, you'll be able to design intelligent control logic for ANY system.

Concept Block (15 min)

What is a Finite State Machine?

Simple definition: A system that can be in ONE of a FINITE number of STATES, and transitions between states based on INPUTS.

Real-world analogy - Traffic Light:

States: RED, YELLOW, GREEN (3 states)

Transitions:

RED → (timer expires) → GREEN

GREEN → (timer expires) → YELLOW

YELLOW → (timer expires) → RED

The traffic light "remembers" which state it's in and transitions based on a timer (input).

FSM Components

Every FSM has 5 parts:

1. States - Possible conditions the system can be in
2. Inputs - External signals that trigger transitions
3. Outputs - Signals produced based on current state
4. Transitions - Rules for moving between states
5. Initial State - Where the FSM starts

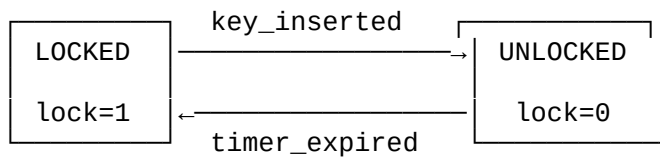
Example - Door Controller:

1. States: LOCKED, UNLOCKED
2. Inputs: key_inserted, timer_expired
3. Outputs: door_lock (1=locked, 0=unlocked)

4. Transitions:
LOCKED → (key_inserted) → UNLOCKED
UNLOCKED → (timer_expired) → LOCKED
5. Initial: LOCKED
-

State Diagram Notation

Visual representation of FSM:



Components:

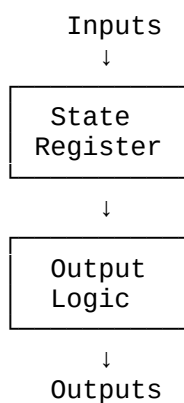
- **Circles/Boxes** = States
 - **Arrows** = Transitions
 - **Labels on arrows** = Input conditions
 - **Text in states** = Outputs
-

Moore vs Mealy Machines

Two types of FSMs based on output generation:

Moore Machine

Output depends **ONLY** on current state.



Characteristics:

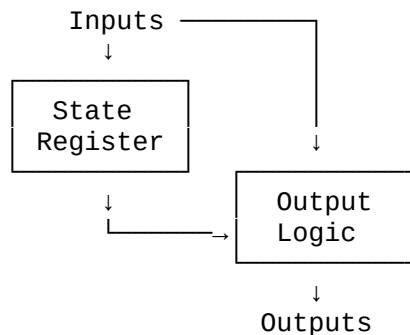
- Outputs are **synchronized** with clock (stable)
- Outputs change **only on state transitions**
- Generally **safer** (no glitches)
- May need **more states** to achieve desired behavior

Example - Traffic Light (Moore):

State: RED → Output: red_light=1, green_light=0
State: GREEN → Output: red_light=0, green_light=1
State: YELLOW → Output: red_light=0, green_light=0, yellow_light=1

Mealy Machine

Output depends on current state AND current inputs.



Characteristics:

- Outputs can change **immediately** when inputs change
- **Faster response** (no need to wait for next clock)
- Can be **more compact** (fewer states needed)
- Risk of **glitches** (outputs change asynchronously)

Example - Sequence Detector (Mealy):

Detecting "101" pattern:

- In any state, if input=1 and next expected, output=1
 - Output changes immediately with input, doesn't wait for next state
-

Comparison Table

Feature	Moore	Mealy
Output depends on	State only	State + Input
Output changes	On clock edge	Immediately with input
Glitches	No (synchronized)	Possible
Response time	Slower (1 clock delay)	Faster (combinational)
Number of states	More states	Fewer states
Typical use	Control sequences, protocols	Pattern detection, fast response
Easier to design	Yes (clearer)	No (more complex)

When to Use Which?

Use Moore when:

- Safety and stability are critical
- You need glitch-free outputs
- Debugging/verification is important

- Protocol implementation (UART, SPI)
- Control sequences (state machines in CPUs)

Use Mealy when:

- Fast response is critical
- You want to minimize states
- Outputs are already registered downstream
- Pattern recognition (sequence detectors)

In practice: Moore is more common in industry because it's safer and easier to verify.

Code & Practical Block (15 min)

The 3-Block Coding Style (Industry Standard)

Professional FSM design uses 3 separate blocks:

```
always @(posedge clk or posedge rst)
    // Block 1: State Register (Sequential)
    // Updates current state on clock edge

always @(*)
    // Block 2: Next State Logic (Combinational)
    // Determines next state based on current state and inputs

always @(*)
    // Block 3: Output Logic (Combinational)
    // Generates outputs based on current state (Moore)
    // or current state + inputs (Mealy)
```

Why 3 blocks?

-  Clear separation of concerns
 -  Easy to debug (each block has one job)
 -  Prevents synthesis issues (latches, etc.)
 -  Industry standard (readable by any engineer)
 -  Easy to verify/simulate
-

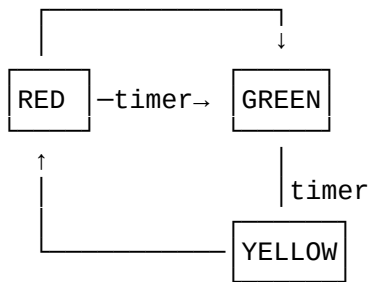
Example 1: Simple Traffic Light (Moore Machine)

Specification:

- 3 states: RED, GREEN, YELLOW
 - Timer input: when high, advance to next state
 - Outputs: red, yellow, green lights (only one at a time)
 - Sequence: RED → GREEN → YELLOW → RED
-

State Diagram:

timer



Verilog Implementation:

```

// traffic_light.v - Simple Traffic Light Controller (Moore FSM)
module traffic_light (
    input wire clk,
    input wire rst,
    input wire timer,      // Timer signal (1 = time to transition)
    output reg red,
    output reg yellow,
    output reg green
);

// State encoding
localparam RED    = 2'b00;
localparam GREEN  = 2'b01;
localparam YELLOW = 2'b10;

// State registers
reg [1:0] current_state;
reg [1:0] next_state;

// Block 1: State Register (Sequential)
always @(posedge clk or posedge rst) begin
    if (rst)
        current_state <= RED;      // Start at RED
    else
        current_state <= next_state;
end

// Block 2: Next State Logic (Combinational)
always @(*) begin
    case (current_state)
        RED: begin
            if (timer)
                next_state = GREEN;
            else
                next_state = RED;
        end

        GREEN: begin
            if (timer)
                next_state = YELLOW;
            else
                next_state = GREEN;
        end

        YELLOW: begin
            if (timer)
                next_state = RED;
            else
                next_state = YELLOW;
        end
    endcase
end

```

```

        next_state = YELLOW;
    end

    default: next_state = RED; // Safe default
endcase
end

// Block 3: Output Logic (Combinational - Moore)
always @(*) begin
    // Default: all lights off
    red = 0;
    yellow = 0;
    green = 0;

    case (current_state)
        RED:    red = 1;
        GREEN:  green = 1;
        YELLOW: yellow = 1;
        default: red = 1; // Safe default
    endcase
end

endmodule

```

Line-by-Line Explanation

Lines 12-14: State Encoding

```

localparam RED    = 2'b00;
localparam GREEN  = 2'b01;
localparam YELLOW = 2'b10;

```

- Assigns binary values to state names
- 2 bits can represent 4 states (we use 3)
- Makes code readable

Lines 17-18: State Registers

```

reg [1:0] current_state;
reg [1:0] next_state;

```

- `current_state`: What state are we in NOW?
- `next_state`: What state are we going to NEXT?

Block 1 (Lines 21-27): State Register

```

always @(posedge clk or posedge rst) begin
    if (rst)
        current_state <= RED;
    else
        current_state <= next_state;
end

```

This is SEQUENTIAL logic:

- On reset: Go to RED state
- On clock edge: Update `current_state` to `next_state`

- This is the ONLY sequential block (flip-flops)

Block 2 (Lines 30-52): Next State Logic

```
always @(*) begin
    case (current_state)
        RED: begin
            if (timer)
                next_state = GREEN;
            else
                next_state = RED;
            end
        // ...
    endcase
end
```

This is COMBINATIONAL logic:

- `always @(*)` = sensitive to ALL signals in the block
- **Decision logic:** "If we're in RED and timer expires, go to GREEN"
- Uses blocking assignment = (combinational)
- Must assign `next_state` in ALL cases (prevents latches)

Block 3 (Lines 55-68): Output Logic

```
always @(*) begin
    red = 0;
    yellow = 0;
    green = 0;

    case (current_state)
        RED:    red = 1;
        GREEN:  green = 1;
        YELLOW: yellow = 1;
    endcase
end
```

This is COMBINATIONAL logic (Moore):

- Outputs depend ONLY on `current_state`
- Default all outputs to 0, then set the active one
- This prevents latches (every path assigns every output)

Critical FSM Coding Rules

Rule 1: Always Assign Next State

```
// WRONG - next_state not assigned in all paths
always @(*) begin
    case (current_state)
        RED: if (timer) next_state = GREEN;
        // Missing: what if timer=0?
    endcase
end
```

```
// CORRECT
always @(*) begin
```

```

    case (current_state)
        RED: begin
            if (timer)
                next_state = GREEN;
            else
                next_state = RED; // Explicitly stay in RED
            end
        endcase
    end
end

```

Rule 2: Include Default Case

```

always @(*) begin
    case (current_state)
        RED:    next_state = GREEN;
        GREEN:  next_state = YELLOW;
        YELLOW: next_state = RED;
        default: next_state = RED; // Handles invalid states
    endcase
end

```

Rule 3: Default All Outputs

```

always @(*) begin
    // Default ALL outputs first
    red = 0;
    yellow = 0;
    green = 0;

    // Then set specific outputs
    case (current_state)
        RED: red = 1;
        // ...
    endcase
end

```

Testbench: Traffic Light

```

// tb_traffic_light.v
module tb_traffic_light;

    reg clk, rst, timer;
    wire red, yellow, green;

    traffic_light uut (
        .clk(clk),
        .rst(rst),
        .timer(timer),
        .red(red),
        .yellow(yellow),
        .green(green)
    );

    // Clock generator
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
end

```

```

// Test stimulus
initial begin
    $dumpfile("traffic_light.vcd");
    $dumpvars(0, tb_traffic_light);

    $display("Time | rst | timer | R | Y | G | State");
    $display("-----");
    $monitor("%4t | %b | %b | %b | %b | %b | %s",
        $time, rst, timer, red, yellow, green,
        (uut.current_state == 2'b00) ? "RED" :
        (uut.current_state == 2'b01) ? "GREEN" :
        (uut.current_state == 2'b10) ? "YELLOW" : "???");

    // Initialize
    rst = 1; timer = 0;
    #10;
    rst = 0;
    #10;

    // Cycle through states with timer pulses
    $display("\n--- RED → GREEN ---");
    timer = 1; #10;
    timer = 0; #10;

    $display("\n--- GREEN → YELLOW ---");
    timer = 1; #10;
    timer = 0; #10;

    $display("\n--- YELLOW → RED ---");
    timer = 1; #10;
    timer = 0; #10;

    $display("\n--- Complete Cycle ---");
    timer = 1; #10; // RED → GREEN
    timer = 0; #10;
    timer = 1; #10; // GREEN → YELLOW
    timer = 0; #10;
    timer = 1; #10; // YELLOW → RED
    timer = 0; #10;

    $finish;
end
endmodule

```

Expected Output

Time		rst		timer		R		Y		G		State	
0		1		0		1		0		0		RED	← Reset
10		0		0		1		0		0		RED	← Stay RED
20		0		1		1		0		0		RED	← timer=1
25		0		1		0		0		1		GREEN	← Transition!
30		0		0		0		0		1		GREEN	
40		0		1		0		0		1		GREEN	
45		0		1		0		1		0		YELLOW	← Transition!
50		0		0		0		1		0		YELLOW	
60		0		1		0		1		0		YELLOW	
65		0		1		1		0		0		RED	← Back to RED!

Compile and Test

```
iverilog -o sim/traffic_light traffic_light.v tb_traffic_light.v  
vvp sim/traffic_light  
gtkwave traffic_light.vcd
```

Mini Task (8 min)

Build a simple 2-state door lock FSM:

States: LOCKED, UNLOCKED

Input: key (1=key inserted, 0=no key)

Output: lock (1=locked, 0=unlocked)

Behavior:

- Start in LOCKED state
- When key=1, transition to UNLOCKED
- When key=0 (in UNLOCKED), transition back to LOCKED

Use the 3-block style!

Build the traffic light FSM, test it, and show me the output!

Once verified, we'll move to **Session 2: Complex FSMs and Pattern Detection** (sequence detectors, Mealy machines).

This is where FSMs get REALLY powerful! 🚀